



Fig. 4: Block diagram of SOA

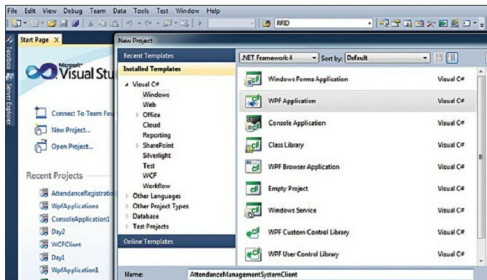


Fig. 5: WPF application selection

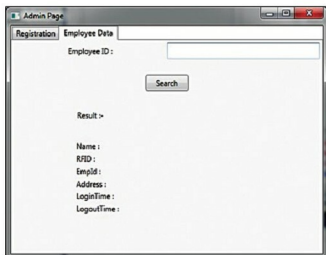


Fig. 6: UI for Employee Data tab

2. **Service layer (connecting layer).** This layer provides an abstraction between client-side and server-side. It follows service oriented architecture (SOA) by using Windows communication foundation (WCF).

3. **Data-access layer (server-side).** In this layer, all data retrieval and update logic is given, which persists and materialises the data in the database. This layer makes the data available to our application. Here, we will use ADO.Net entity framework.

4. **Database.** Data is stored and retrieved by the user via data-access layer.

Since we are developing a UI on

Microsoft environment, the better option would be to use WPF as the graphical-presentation system. We would be following an SOA to make the connecting layer act as a service layer for client and server programs. Service-oriented applications can be built in .Net by using a set of runtime APIs released by Microsoft by the name WCF. As the name suggests, WCF

is a tool used for the deployment of SOA (Fig. 4).

Features and objectives of client UI

1. The UI contains a page for the system administrator to access.
2. The page contains two tabs: Registration and Employee Data
3. Registration tab contains fields to enter user name, user address, serial port information and a Register button to register and save information of a newly-registered person in the database.
4. Employee Data tab contains a field for the administrator to enter

the employee ID. If a single detail of the person is found in the database, all details of that person would be displayed.

Designing client-side application

1. Open Visual Studio 2010 in administrator mode (if not in admin mode by default). Select New Project to create a new application.

2. A dialog box will open. In that, under Visual C#, select Windows and then from the middle pane, select WPF application and name it AttendanceManagementSystemClient as shown in Fig. 5. Name the solution AttendanceClientSide. A solution of the same name opens with a WPF project inside it.

3. Now, we have a window where we want a layout to put different controls at different places. To do that, we will design the layout for it. MainWindow.xaml code (included in EFY DVD of this month's issue) generates our desired layout for the client UI.

Copying and pasting this code in MainWindow.xaml file generates the two UIs shown in Fig. 1 and Fig. 6.

After the above procedure is successfully followed, building the project will give some error related to undefined functions. Go to Registration UI and double-click on TextBlock 'Serial Port number' under PortInfo, and Register and Search buttons. This will create an empty function definition for these. There should be zero errors now.

In MainWindow.xaml, we will create the main UI for our software. We start with creating a window tag in lines one through four of MainWindow.xaml code. This code is under namespace AttendanceManagementSystemClient. The created .xaml file also has to use two namespaces: `http://schemas.microsoft.com/winx/2006/xaml/presentation` and `http://schemas.microsoft.com/winx/2006/xaml`.

Here, we have given the title Admin Page to the window, and the height and width of the window are 400 and 525 device-independent units, respectively. From now on, the entire UI will be created inside this window tag.